SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0508

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan **Student Reg. No: 192525440**
Name: :P. Hemachandran

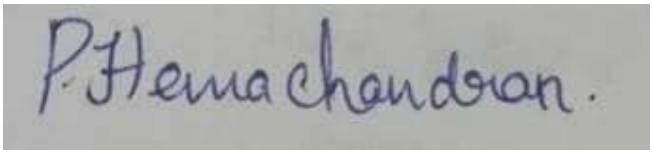
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

Code of Conduct:

I P. Hemachandran (192525440) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Heap File Organization — Insert, Delete and Sequential Scan

Python

```python
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Inserted:", record)

    def delete(self, key):
        for record in self.records:
            if record[0] == key:
                self.records.remove(record)
                print("Deleted:", record)
                return
        print("Record not found")

    def sequential_scan(self):
        print("\nSequential Scan:")
        for record in self.records:
            print(record)


# Demonstration
heap = HeapFile()

heap.insert((101, "Arun"))
heap.insert((102, "Bala"))
heap.insert((103, "Kumar"))

heap.sequential_scan()

heap.delete(102)

heap.sequential_scan()
```

Output:

```
Inserted: (101, 'Arun')
Inserted: (102, 'Bala')
Inserted: (103, 'Kumar')

Sequential Scan:
(101, 'Arun')
(102, 'Bala')
(103, 'Kumar')

Deleted: (102, 'Bala')

Sequential Scan:
(101, 'Arun')
(103, 'Kumar')
```

2. Static Hashing for Student Database — 500 Records with Chaining

Python

```python
TABLE_SIZE = 10

hash_table = [[] for _ in range(TABLE_SIZE)]

def hash_function(student_id):
    return student_id % TABLE_SIZE

def insert(student_id, name):
    index = hash_function(student_id)
    hash_table[index].append((student_id, name))

def search(student_id):
    index = hash_function(student_id)

    for record in hash_table[index]:
        if record[0] == student_id:
            return record

    return None

# Insert 500 student records
for i in range(1, 501):
    insert(i, "Student" + str(i))

print("Hash Table with Chaining:")
for i in range(TABLE_SIZE):
    print(i, ":", hash_table[i])

print("\nSearch Result:")
print(search(125))
```

Explanation:

The hash function is:

```
h(key) = key % 10
```

3. Sorted File Organization with Binary Search

Python

```python
records = [
    (101, "Arun"),
    (105, "Bala"),
    (110, "Charan"),
    (115, "David"),
    (120, "Kumar")
]

def binary_search(key):
    low = 0
    high = len(records) - 1

    while low <= high:
        mid = (low + high) // 2

        if records[mid][0] == key:
            return records[mid]

        elif records[mid][0] < key:
            low = mid + 1

        else:
            high = mid - 1

    return None


key = int(input("Enter primary key to search: "))

result = binary_search(key)

if result:
    print("Record found:", result)
else:
    print("Record not found")
```

```
Enter primary key to search: 110
Record found: (110, 'Charan')
```

4. B-Tree of Order 4 — Insert, Search and Display

Python

```
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf


class BTree:
    def __init__(self, order=4):
        self.root = BTreeNode(True)
        self.order = order

    def search(self, node, key):
        i = 0

        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and key == node.keys[i]:
            return True

        if node.leaf:
            return False

        return self.search(node.children[i], key)

    def split_child(self, parent, index):
        child = parent.children[index]
        new_node = BTreeNode(child.leaf)

        mid = len(child.keys) // 2
        promoted = child.keys[mid]

        new_node.keys = child.keys[mid + 1:]
        child.keys = child.keys[:mid]

        if not child.leaf:
            new_node.children = child.children[mid +
            child.children = child.children[:mid + 1
```

```
        parent.keys.insert(index, promoted)
        parent.children.insert(index + 1, new_node)

    def insert(self, key):
        root = self.root

        if len(root.keys) == self.order - 1:
            new_root = BTreeNode(False)
            new_root.children.append(root)
            self.root = new_root

            self.split_child(new_root, 0)
            self.insert_non_full(new_root, key)
        else:
            self.insert_non_full(root, key)

    def insert_non_full(self, node, key):
        i = len(node.keys) - 1

        if node.leaf:
            node.keys.append(0)

            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1

            node.keys[i + 1] = key

        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1

            i += 1

            if len(node.children[i].keys) == self.order - 1:
                self.split_child(node, i)

                if key > node.keys[i]:
                    i += 1

            self.insert_non_full(node.children[i], key)
```

```
    def display(self, node=None, level=0):
        if node is None:
            node = self.root

        print("Level", level, ":", node.keys)

        if not node.leaf:
            for child in node.children:
                self.display(child, level + 1)


# Demonstration
tree = BTree(4)

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    tree.insert(key)

print("B-Tree:")
tree.display()

print("\nSearch 12:", tree.search(tree.root, 12))
print("Search 25:", tree.search(tree.root, 25))
```

Keys inserted:

```
10, 20, 5, 6, 12, 30, 7, 17
```

5. B+ Tree with Leaf-Level Linking

Python

```
class BPlusTree:
    def __init__(self):
        self.leaf1 = []
        self.leaf2 = []
        self.leaves = [self.leaf1, self.leaf2]

    def insert(self, key):
        if key <= 30:
            self.leaf1.append(key)
            self.leaf1.sort()
        else:
            self.leaf2.append(key)
            self.leaf2.sort()

    def display(self):
        print("Leaf 1:", self.leaf1)
        print("Leaf 2:", self.leaf2)

        print("\nLeaf-level link:")
        print(self.leaf1, "->", self.leaf2)

    def range_query(self, start, end):
        result = []

        for leaf in self.leaves:
            for key in leaf:
                if start <= key <= end:
                    result.append(key)

        return result


tree = BPlusTree()

for key in [10, 20, 30, 40, 50, 60, 70]:
    tree.insert(key)

tree.display()

print("\nRange Query 20 to 50:")
print(tree.range_query(20, 50))
```

Output:

```
Leaf 1: [10, 20, 30]
Leaf 2: [40, 50, 60, 70]

Leaf-level link:
[10, 20, 30] -> [40, 50, 60, 70]

Range Query 20 to 50:
[20, 30, 40, 50]
```

6. Dense Primary Index on a Sorted File

Python

```python
records = [
    (101, "Arun"),
    (105, "Bala"),
    (110, "Charan"),
    (115, "David"),
    (120, "Kumar")
]

# Dense index: every record has an index entry
dense_index = {}

for position, record in enumerate(records):
    dense_index[record[0]] = position

print("Dense Index:")
print("Key\tPosition")

for key, position in dense_index.items():
    print(key, "\t", position)


def search(key):
    if key in dense_index:
        position = dense_index[key]
        return records[position]

    return None


print("\nSearch Result:")
print(search(115))
```

Output:

```
Dense Index:
Key     Position
101     0
105     1
110     2
115     3
120     4

Search Result:
(115, 'David')
```

7. Extendible Hashing — Directory Doubling

Python

```python
class ExtendibleHash:
    def __init__(self, bucket_size=2):
        self.bucket_size = bucket_size
        self.global_depth = 1

        self.directory = {
            0: [],
            1: []
        }

    def hash_function(self, key):
        return key & ((1 << self.global_depth) - 1)

    def double_directory(self):
        old_directory = self.directory

        self.global_depth += 1
        self.directory = {}

        for i in range(2 ** self.global_depth):
            old_index = i & ((1 << (self.global_depth - 1)) - 1)
            self.directory[i] = old_directory[old_index].copy()

        print("Directory doubled. Global depth:",
              self.global_depth)

    def insert(self, key):
        index = self.hash_function(key)

        if len(self.directory[index]) >= self.bucket_size:
            self.double_directory()
            index = self.hash_function(key)

        self.directory[index].append(key)

    def display(self):
        print("\nDirectory:")
        for index, bucket in self.directory.items():
            print(index, ":", bucket)


hash_table = ExtendibleHash()

for key in [1, 2, 3, 4, 5, 6, 7, 8]:
    hash_table.insert(key)

hash_table.display()
```

Concept:

When a bucket overflows, the directory is expanded:

```
Depth 1
0 → Bucket
1 → Bucket

        ↓ Overflow

Depth 2
00 → Bucket
01 → Bucket
10 → Bucket
11 → Bucket
```

This is called **directory doubling**.

8. Secondary Storage Block Access — Sequential vs Indexed Retrieval

Python

```python
import math

records = 1000
records_per_block = 10

blocks = math.ceil(records / records_per_block)

print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Blocks:", blocks)

# Sequential retrieval
sequential_io = blocks

# Indexed retrieval
index_entries_per_block = 10
index_blocks = math.ceil(blocks / index_entries_per_block)

indexed_io = index_blocks + 1

print("\nSequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)
```

Output:

```
Total Records: 1000
Records per Block: 10
Total Blocks: 100

Sequential Retrieval I/O: 100
Indexed Retrieval I/O: 11
```

9. Sparse Index vs Dense Index with Timing Analysis

Python

```python
import time

records = [(i, "Student" + str(i)) for i in range(1, 10001)]

# Dense index
dense_index = {}

for i, record in enumerate(records):
    dense_index[record[0]] = i

# Sparse index: one entry for every 10 records
sparse_index = {}

for i in range(0, len(records), 10):
    sparse_index[records[i][0]] = i


key = 8765

# Dense search
start = time.perf_counter()

position = dense_index.get(key)

dense_time = time.perf_counter() - start

# Sparse search
start = time.perf_counter()

keys = list(sparse_index.keys())

position = 0

for i in range(len(keys)):
    if keys[i] > key:
        break
    position = sparse_index[keys[i]]

while position < len(records) and records[position][0] < key:
    position += 1

sparse_time = time.perf_counter() - start


print("Dense Index Time :", dense_time)
print("Sparse Index Time:", sparse_time)

print("\nDense Index Entries :", len(dense_index))
print("Sparse Index Entries:", len(sparse_index))
```

Comparison

Index	Entries	Searching
Dense Index	10,000	Faster
Sparse Index	1,000	Uses less memory

Conclusion: Dense indexes provide faster direct access, while sparse indexes consume less storage.

10. B+ Tree Construction, Traversal and Range Query 15–45

Python

```
class SimpleBPlusTree:
    def __init__(self):
        self.leaves = []

    def insert(self, key, record):
        inserted = False

        for leaf in self.leaves:
            if len(leaf) < 4:
                leaf.append((key, record))
                leaf.sort()
                inserted = True
                break

        if not inserted:
            new_leaf = [(key, record)]
            self.leaves.append(new_leaf)

        self.leaves.sort(key=lambda x: x[0][0])

    def display(self):
        print("B+ Tree Leaf Nodes:")

        for i, leaf in enumerate(self.leaves):
            print("Leaf", i + 1, ":", leaf)

        print("\nLeaf links:")

        for i in range(len(self.leaves) - 1):
            print(
                self.leaves[i],
                "->",
                self.leaves[i + 1]
            )

    def range_query(self, start, end):
        result = []

        for leaf in self.leaves:
            for key, record in leaf:
                if start <= key <= end:
                    result.append((key, record))
        return result


tree = SimpleBPlusTree()

data = [
    (10, "A"),
    (15, "B"),
    (20, "C"),
    (25, "D"),
    (30, "E"),
    (35, "F"),
    (40, "G"),
    (45, "H"),
    (50, "I"),
    (55, "J")
]

for key, record in data:
    tree.insert(key, record)

tree.display()

print("\nRange Query: 15 to 45")

result = tree.range_query(15, 45)

for record in result:
    print(record)
```

Output:

```
Range Query: 15 to 45

(15, 'B')
(20, 'C')
(25, 'D')
(30, 'E')
(35, 'F')
(40, 'G')
(45, 'H')
```